

Constructors in Java

01 May 2026 06:51

In Java, a constructor is a special method used to initialize object, when a class is created.

What is a Constructor in Java?

A constructor in Java is a block of code like the method. It is called when an instance of a class is created. At the time of calling constructor, memory for the object is allocated in the memory. It is a special type of method which is used to initialize the object.

Every time when we create an object by using the new keyword. It calls a default constructor. If there is no constructor available in the class. In such case Java compiler provides a default constructor by default.

Note: It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class because Java compiler creates a default constructor if a class does not have any.

Rules for Creating Java Constructor

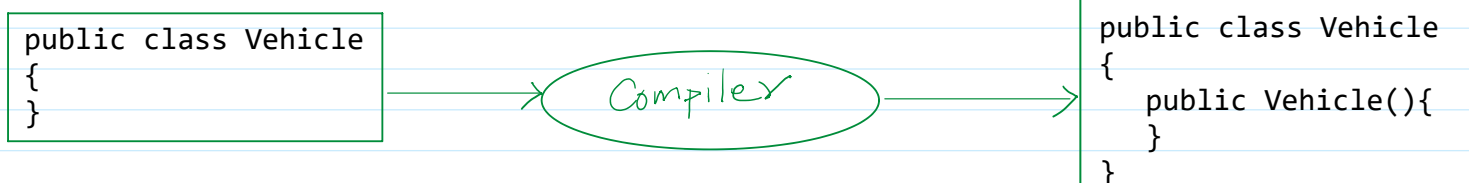
1. Constructor name must be the same as its class name.
2. The constructor must have no explicit return type.
3. A constructor cannot be abstract, static, final, or synchronized.

Note: We can use access modifiers while declaring a constructor. It controls the object creation. A constructor may be private, protected, public, or default.

Types of Java Constructors

There are two types of constructors in Java:

1. **Default Constructor(No-arg constructor):** The default constructor does not take any parameters and is used to initialize an object with default values. If no constructor is defined, Java automatically provides a default constructor.
2. **Parameterized Constructor:** The parameterized constructor accepts the parameters and is used to initialize an object with specific values at the time of object creation.
 - a. **Copy constructor:** A copy constructor is a special type of constructor that is commonly utilized to create a new object by copying the values of an existing object of the same class.



Purpose of a default constructor

The default constructor is used to provide the values to the object like zero, null etc. depending on the type.

package OOP;

```
/**
 * Write a description of class Vehicle here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Vehicle
{
    //instance variables
    int engineNum;
    String model;
    double price;

    public void display(){
        System.out.print("\nEngine number: " + engineNum
            + "\n Model: " + model+
            "\nPrice; " + price);
    }
}
```

Output:

```
Engine number: 0
Model: null
Price: 0.0
```